

IUT DE VALENCE — UNIVERSITÉ GRENOBLE ALPES

Département Informatique — BUT 1re année

SAÉ S2.03

Configurer un service sur une machine en réseau

*Installer et configurer un serveur pour le
développement web*

RAPPORT FINAL

Apache · PHP · MariaDB · NFS · Symfony · Java

Binôme :

Ghlilech Billal

Karami Chakib

Date :

Juin 2026

Sommaire

1	Rappel de la définition des besoins	2
1.1	Contexte et objectif	2
1.2	Besoins fonctionnels	2
1.3	Architecture cible et conventions	2
2	Compte-rendu des choix et des opérations	3
2.1	Installation du serveur (Debian sans interface graphique)	3
2.2	Configuration du réseau	7
2.3	Accès distant : tunnel SSH / HTTP	10
2.4	Services web et base de données (Apache / PHP / MariaDB)	11
2.5	Pages personnelles des utilisateurs (UserDir)	12
2.6	Base de données : administrateur et isolation	13
2.7	Scripts d'automatisation	14
2.8	Partage NFS	18
2.9	Liaison PHP / base de données (PDO)	21
2.10	Outils de développement Symfony et Composer	21
2.11	Environnement Java	21
3	Méthode de déploiement et tests de validation	21
3.1	Comment ajouter un nouvel utilisateur	21
3.2	Tests de validation	22
3.3	Difficultés rencontrées et solutions	22
3.4	Conclusion et accès d'administration	23
3.5	Accès aux machines	24

1 Rappel de la définition des besoins

1.1 Contexte et objectif

L'objectif de cette SAÉ est de mettre à la disposition des programmeurs d'une organisation un serveur de développement web mutualisé. Ce serveur doit permettre à plusieurs utilisateurs de développer et tester leurs sites web, avec un accès à une base de données, aussi bien en PHP qu'en Java.

Le serveur est une machine virtuelle (VirtualBox) configurée sans interface graphique, administrée à distance, et accessible depuis une machine cliente. Le serveur n'ayant pas d'environnement graphique, deux problématiques se posent et structurent tout le projet : comment l'administrer (réseau, SSH, tunnel) et comment permettre à un utilisateur d'éditer confortablement ses fichiers (partage NFS vers une machine cliente disposant, elle, d'un bureau et d'éditeurs graphiques).

1.2 Besoins fonctionnels

Le cahier des charges impose les fonctionnalités suivantes :

- Un serveur web Apache + PHP, accessible depuis le client à l'adresse `http://192.168.1.1`, présentant une page de bienvenue avec un lien vers la page d'information de PHP (`phpinfo()`).
- Des pages personnelles par utilisateur, à l'URL `http://192.168.1.1/~login`, affichant un message de bienvenue avec le nom de l'utilisateur et la date du jour, ainsi qu'un lien vers le contenu d'une table de sa base de données.
- Un SGBD MySQL/MariaDB avec un compte administrateur, et une base par utilisateur portant son identifiant : chaque utilisateur ne peut accéder qu'à sa propre base (isolation).
- Un script `createDatabase` (compte root) créant la base d'un utilisateur, et un script `createUser` réalisant l'ensemble des opérations de création d'un utilisateur.
- Les utilitaires Symfony et Composer installés dans `/usr/local/bin`, accessibles à tous les utilisateurs.
- Un accès Java aux bases via le connecteur MariaDB (programme `DisplayContacts.java`).
- Un partage NFS : l'arborescence `/home` du serveur est exportée et montée sur `/users` du client, afin qu'un utilisateur édite ses fichiers du serveur depuis le client (l'utilisateur doit avoir le même identifiant (UID) sur les deux machines).

1.3 Architecture cible et conventions

Deux modes d'accès au serveur sont demandés : un réseau interne privé entre client et serveur, et un accès depuis la machine hôte Windows via un tunnel (SSH et HTTP). Les conventions d'adressage et de ports retenues pour l'ensemble du projet sont synthétisées ci-dessous.

Élément	Valeur	Rôle
Serveur (réseau interne)	192.168.1.1	Serveur web, base de données, NFS
Client (réseau interne)	192.168.1.7	Poste de développement, tests, montage NFS
Réseau interne VirtualBox	intnet	Lien privé isolé client ↔ serveur
Tunnel SSH	hôte 2222 → invité 22	Administration du serveur depuis Windows
Tunnel HTTP	hôte 8888 → invité 80	Test du site depuis Windows

2 Compte-rendu des choix et des opérations

2.1 Installation du serveur (Debian sans interface graphique)

Le serveur est installé sous Debian 13 (Trixie). Après plusieurs essais sous Ubuntu présentant des instabilités, Debian a été retenue pour sa stabilité et sa légèreté. Le choix de ne PAS installer d'environnement graphique est demandé : l'installation est plus rapide et reproductible, la machine virtuelle est nettement plus légère, et les outils graphiques sont inutiles sur un serveur car cela consommerait beaucoup trop. À l'étape de sélection des logiciels (tasksel), l'environnement de bureau a donc été décoché et seul le serveur SSH a été retenu.

Le disque a été partitionné manuellement en trois partitions : / (racine), **swap**, et **/home**. Séparer **/home** se justifie particulièrement ici, car c'est cette arborescence qui sera ensuite exportée en NFS ; la placer sur une partition dédiée isole les données des développeurs du système.

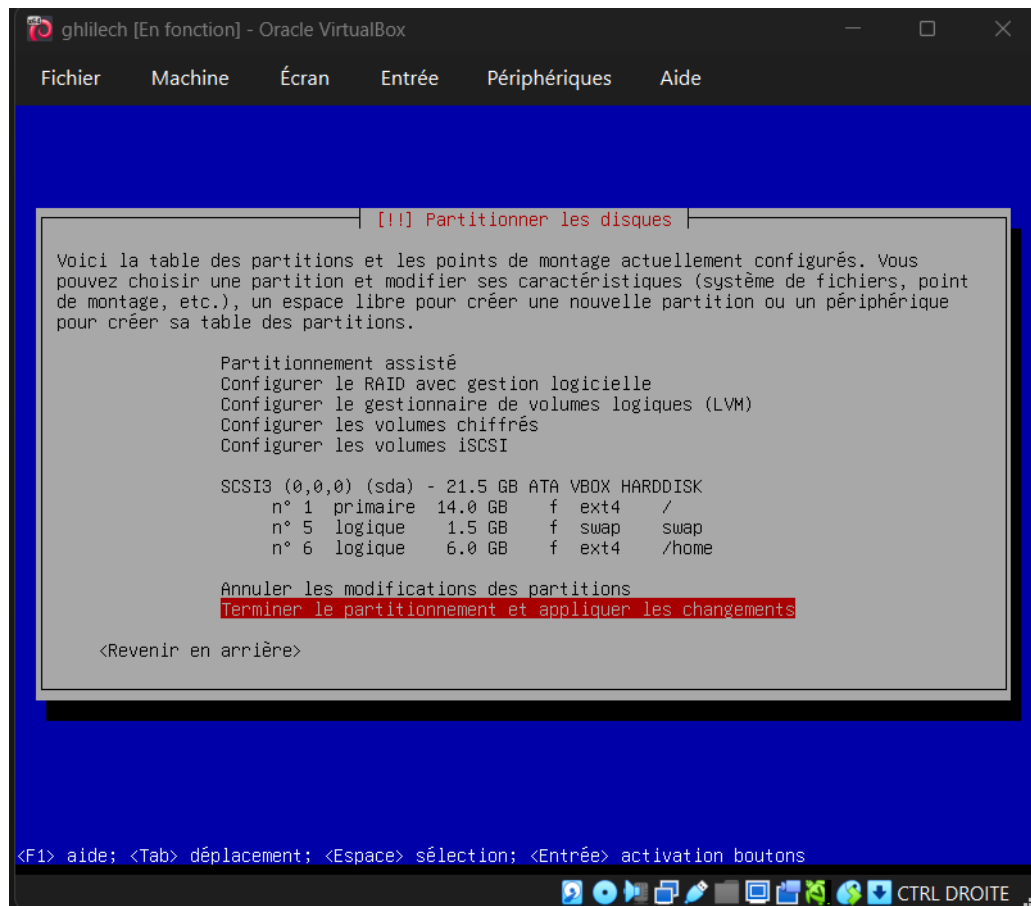


FIGURE 1 – Partitionnement manuel du disque du serveur (/ , swap, /home).

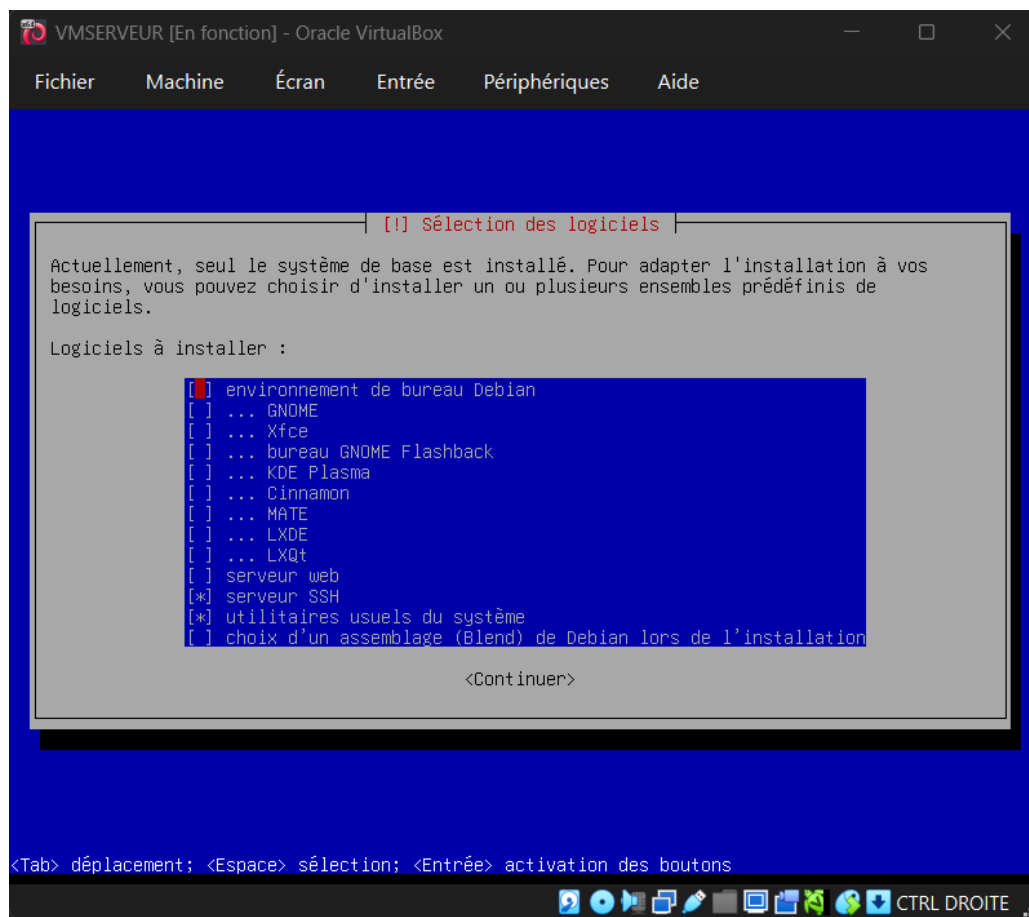


FIGURE 2 – Sélection des logiciels : environnement de bureau décoché, serveur SSH coché.

Le chargeur de démarrage GRUB est installé sur le disque (`/dev/sda`) afin de rendre la machine amorçable.

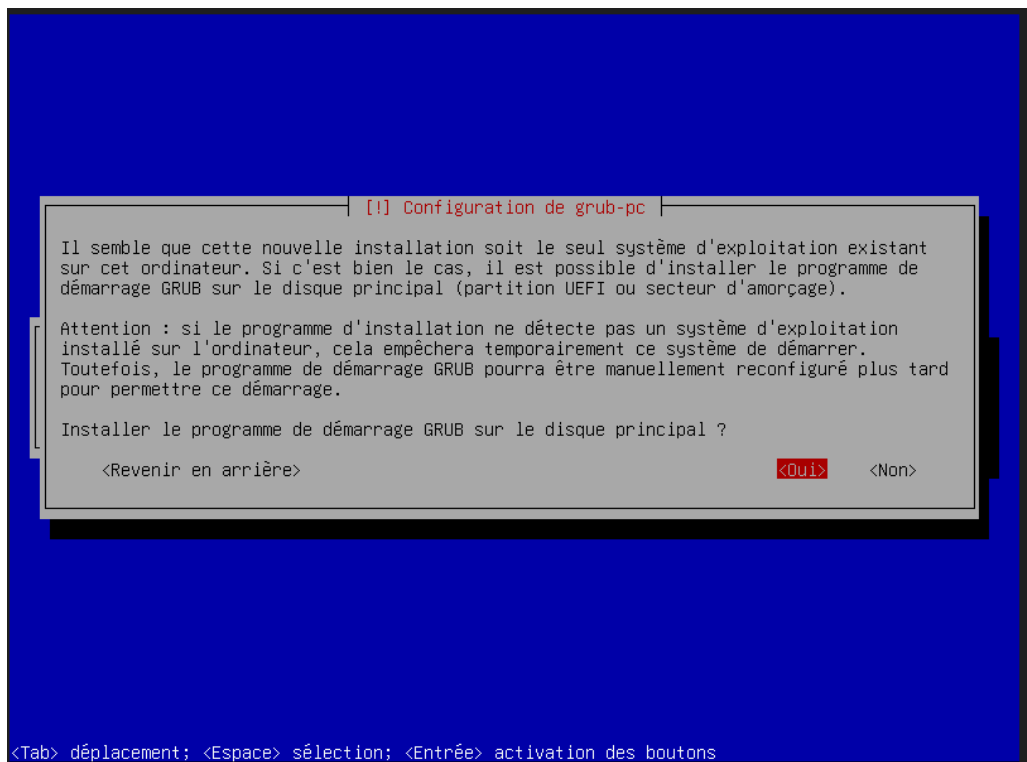


FIGURE 3 – Installation du chargeur de démarrage GRUB sur le disque principal.

Après redémarrage, on se connecte en mode console. La commande `ls /home` permet d'identifier le compte utilisateur créé pendant l'installation (`bilelchakvm`).

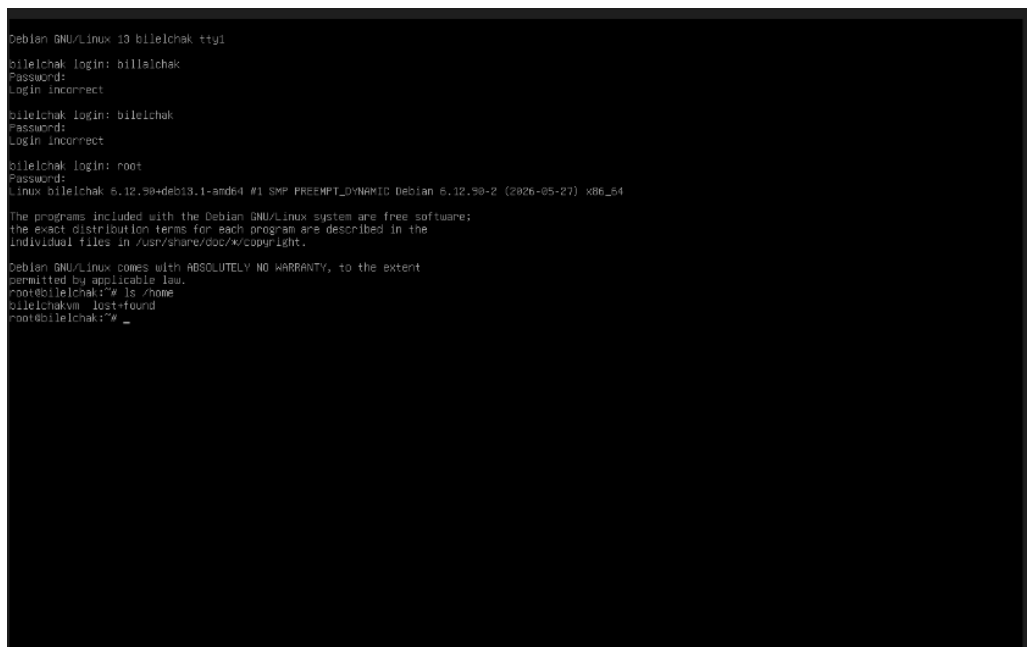


FIGURE 4 – Premier démarrage du serveur en mode texte et identification du compte utilisateur.

On vérifie enfin que le service SSH est actif (`active/running`) et activé au démarrage (`enabled`), en écoute sur le port 22 — ce qui sera indispensable pour le tunnel.

```

Debian GNU/Linux 13 bilechak tty1
bilechak login: root
Password:
Linux bilechak 6.12.90+deb13.1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.90-2 (2026-05-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@bilechak:~# systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-15 13:51:06 CEST; 46s ago
  Invocation: ad8940ce66674ce9af29a10310983191
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 778 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Main PID: 822 (sshd)
       Tasks: 1 (limit: 2391)
      Memory: 2.8M (peak: 3.3M)
         CPU: 48ms
    CGroup: /system.slice/ssh.service
            └─822 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jun 15 13:51:06 bilechak systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jun 15 13:51:06 bilechak sshd[822]: Server listening on 0.0.0.0 port 22.
Jun 15 13:51:06 bilechak sshd[822]: Server listening on :: port 22.
Jun 15 13:51:06 bilechak systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@bilechak:~#

```

FIGURE 5 – Service SSH actif et activé au démarrage, en écoute sur le port 22.

2.2 Configuration du réseau

Chaque machine est équipée de deux cartes réseau, ayant chacune un rôle distinct. La carte 1, en mode NAT, fournit l'accès à Internet, indispensable pour télécharger les paquets via `apt`. La carte 2, en mode Réseau interne (« intnet »), crée un réseau privé et isolé sur lequel le client et le serveur communiquent.

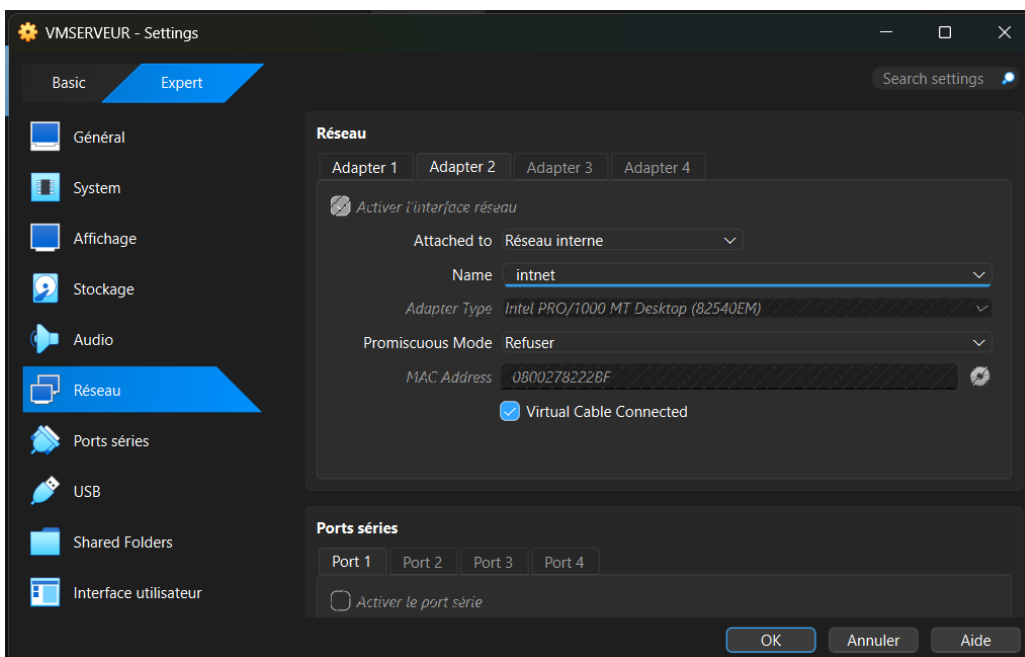


FIGURE 6 – Carte 2 du serveur configurée en Réseau interne « intnet ».

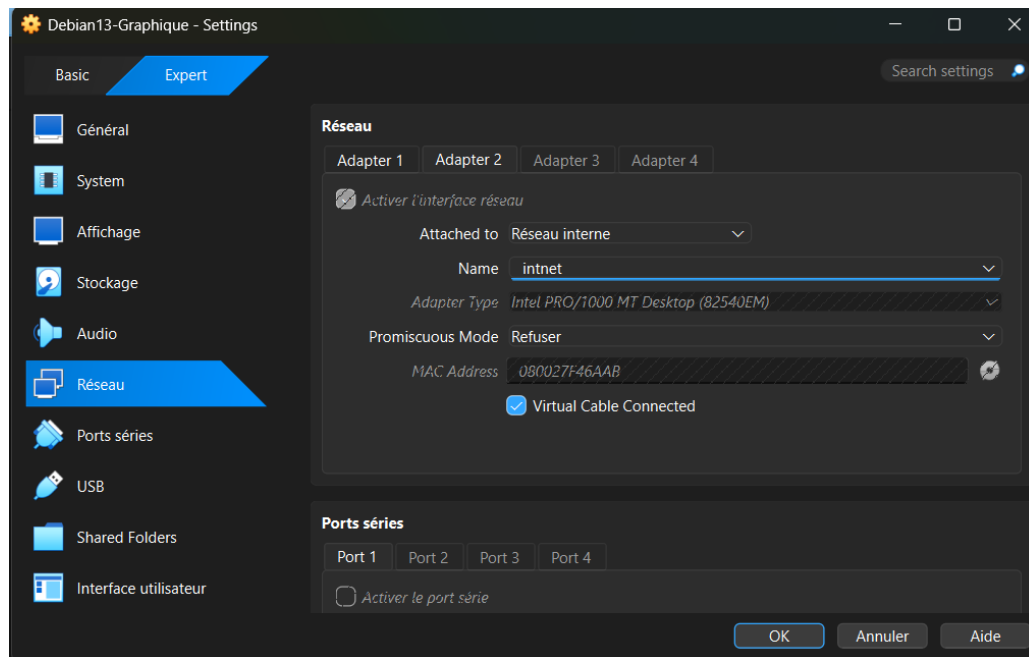


FIGURE 7 – Carte 2 du client configurée sur le même réseau interne « intnet ».

Le serveur reçoit l'adresse statique 192.168.1.1. Une adresse fixe est nécessaire sur un serveur pour que le client sache toujours où le joindre. La configuration se fait dans `/etc/network/interfaces` :

```
auto enp0s8
iface enp0s8 inet static
    address 192.168.1.1
    netmask 255.255.255.0
```

```
root@bilelchak:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:9b:f1:8b brd ff:ff:ff:ff:ff:ff
   altname enx0800279bf18b
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 83256sec preferred_lft 72456sec
   inet6 fd17:625c:f037:2:a00:27ff:fe9b:f18b/64 scope global dynamic mngtmpaddr proto kernel_r
       valid_lft 86315sec preferred_lft 14315sec
   inet6 fd17:625c:f037:2:7a6d:616d:d94:dc20/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86315sec preferred_lft 14315sec
   inet6 fe80::71ad:585e:297d:269/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:82:22:bf brd ff:ff:ff:ff:ff:ff
   altname enx0800278222bf
   inet 192.168.1.1/24 brd 192.168.1.255 scope global enp0s8
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fe82:22bf/64 scope link proto kernel_l1
       valid_lft forever preferred_lft forever
root@bilelchak:~#
```

FIGURE 8 – `ip a` sur le serveur : l'interface `enp0s8` porte bien l'adresse 192.168.1.1/24.

Côté client (Debian avec interface graphique), la configuration directe de `/etc/network/interfaces` échouait systématiquement, car le réseau y est géré par NetworkManager, qui entre en conflit avec ce fichier. L'adresse statique 192.168.1.7 a donc été définie via l'interface

graphique de NetworkManager (méthode IPv4 « Manuel »), ce qui s'est révélé stable et persistant.

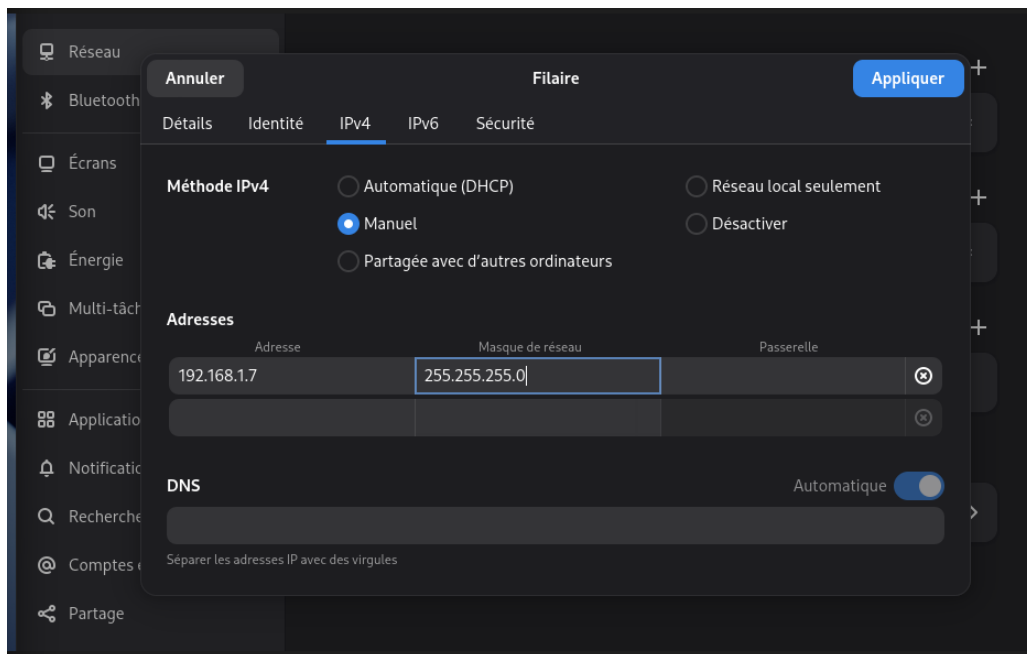


FIGURE 9 – Configuration de l'adresse statique du client (192.168.1.7) via NetworkManager.

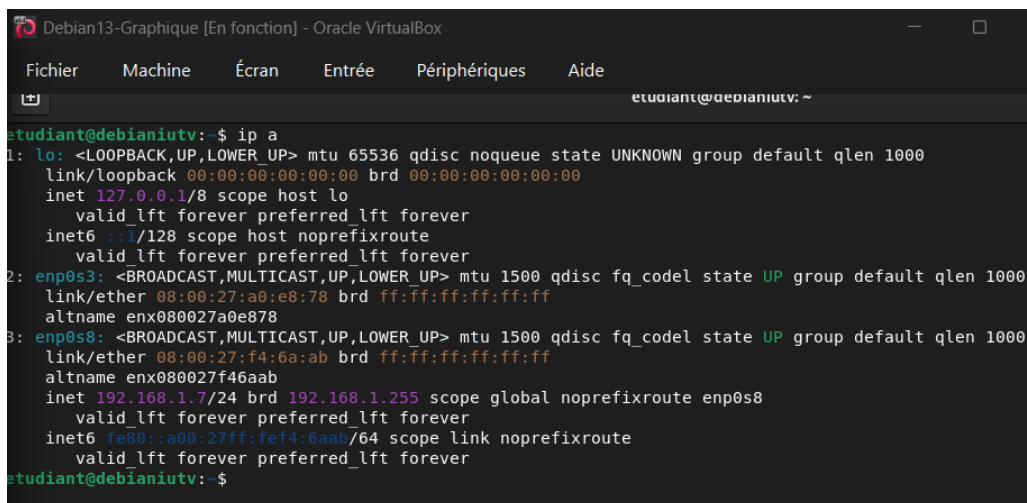


FIGURE 10 – `ip a` sur le client : l'interface `enp0s8` porte bien l'adresse 192.168.1.7/24.

La validation du réseau se fait par un test de connectivité : depuis le client, le ping du serveur 192.168.1.1 répond, confirmant que le réseau privé est pleinement opérationnel.

```

Debian13-Graphique [En fonction] - Oracle VirtualBox
Fichier  Machine  Écran  Entrée  Périphériques  Aide

link/ether 08:00:27:f4:6a:ab brd ff:ff:ff:ff:ff:ff
altname enx080027f46aab
inet 192.168.1.7/24 brd 192.168.1.255 scope global noprefixroute enp0s8
    valid_lft forever preferred_lft forever
inet6 fe80::a00:27ff:fe4:6aab/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
etudiant@debianitv:~$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.23 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.775 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=3.74 ms
64 bytes from 192.168.1.1: icmp_seq=4 ttl=64 time=1.28 ms
64 bytes from 192.168.1.1: icmp_seq=5 ttl=64 time=2.13 ms
64 bytes from 192.168.1.1: icmp_seq=6 ttl=64 time=8.93 ms
64 bytes from 192.168.1.1: icmp_seq=7 ttl=64 time=1.18 ms

```

FIGURE 11 – Test de connectivité réussi : ping du serveur depuis le client (réseau privé opérationnel).

2.3 Accès distant : tunnel SSH / HTTP

Le serveur n'ayant pas d'interface, on met en place un tunnel depuis l'hôte Windows. Dans VirtualBox, sur la carte NAT du serveur, deux redirections de ports sont créées : le port hôte 2222 vers le port 22 du serveur (SSH), et le port hôte 8888 vers le port 80 (HTTP). VirtualBox isolant la VM derrière le NAT, ce tunnel est le seul moyen d'administrer le serveur en SSH depuis Windows et d'y copier-coller du code proprement.

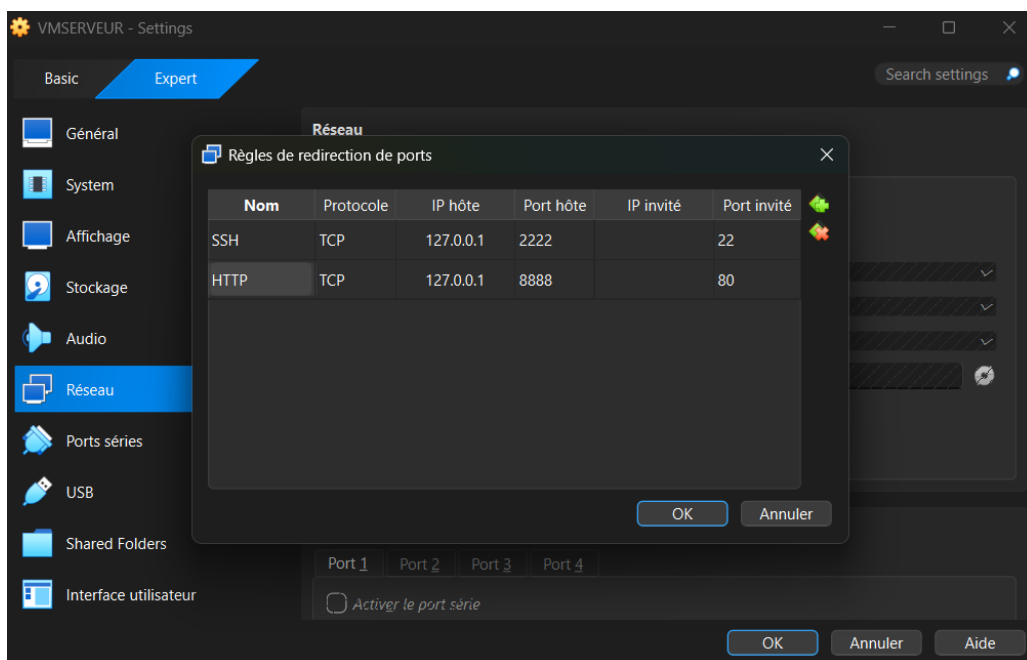


FIGURE 12 – Règles de redirection de ports dans VirtualBox : SSH (2222→22) et HTTP (8888→80).

La connexion depuis le terminal Windows s'établit ainsi :

```
ssh -p 2222 bilelchakvm@localhost
```

```
Microsoft Windows [version 10.0.26200.8655]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Bilel>ssh -p 2222 bilelchakvm@localhost
bilelchakvm@localhost's password:
Linux bilelchak 6.12.90+deb13.1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.90-2 (2026-05-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 15 14:58:34 2026 from 10.0.2.2
bilelchakvm@bilelchak:~$ |
```

FIGURE 13 – Connexion SSH au serveur depuis l'hôte Windows, via le tunnel (port 2222).

2.4 Services web et base de données (Apache / PHP / MariaDB)

L'ensemble des paquets nécessaires est installé en une commande :

```
apt install apache2 php libapache2-mod-php php-mysql php-intl \
mariadb-server openjdk-21-jdk -y
```

Le paquet `php-intl` mérite une attention particulière : le fichier `displayContacts.php` fourni utilise la classe `IntlDateFormatter` pour afficher la date en français. Sans ce paquet, la page renvoie une erreur fatale (`Class "IntlDateFormatter" not found`). Ce point a d'ailleurs fait l'objet d'un message du responsable de la SAÉ.

Une fois l'installation faite, on valide PHP avec une page `phpinfo()` : la version 8.4.21 s'affiche correctement, ce qui confirme qu'Apache exécute bien le PHP.

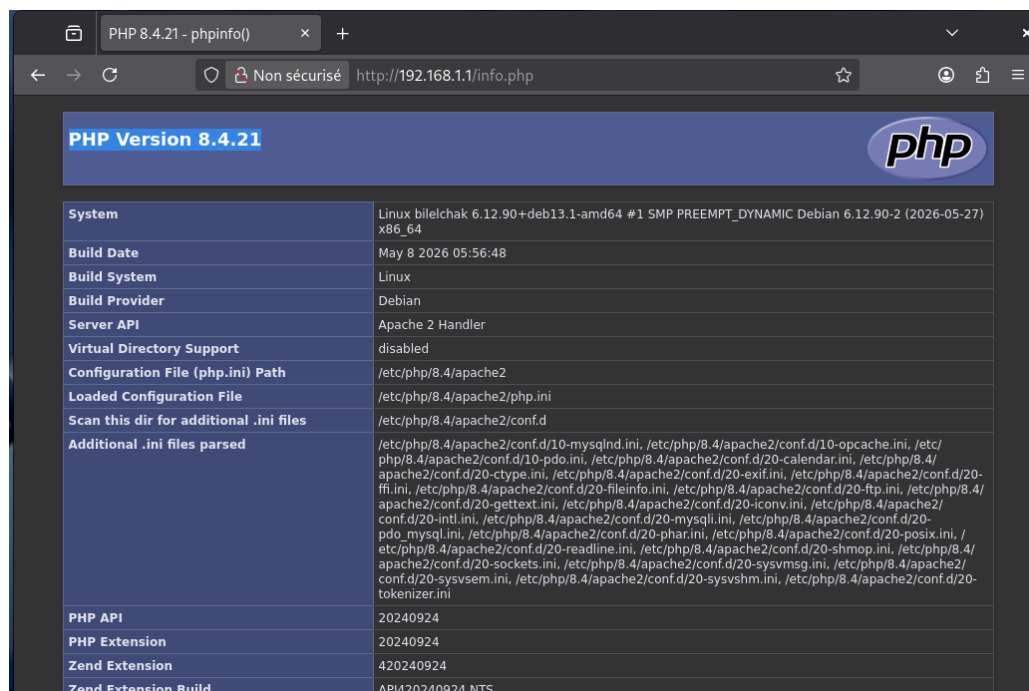


FIGURE 14 – Page `phpinfo()` : PHP 8.4.21 fonctionne via Apache (Server API : Apache 2 Handler).

La page d'accueil par défaut d'Apache est remplacée par une page de bienvenue contenant un lien vers la page `phpinfo()`, conformément au cahier des charges.



FIGURE 15 – Page d'accueil du serveur (`http://192.168.1.1`) avec lien vers `phpinfo()`.

2.5 Pages personnelles des utilisateurs (UserDir)

Le module Apache `userdir` est activé pour permettre l'accès aux pages personnelles via l'URL `~login`. Par défaut, Debian interdit l'exécution de PHP dans les dossiers personnels : il faut commenter entièrement le bloc `<IfModule mod_userdir.c>` du fichier `/etc/apache2/mods-available/php8.4.conf`, puis redémarrer Apache.

```
a2enmod userdir
nano /etc/apache2/mods-available/php8.4.conf # commenter le bloc userdir
systemctl restart apache2
```

Un point d'attention important concerne les droits : le répertoire personnel doit être en `chmod 755`, car Apache (qui s'exécute sous l'utilisateur `www-data`) doit pouvoir le traverser pour atteindre `public_html`. Sans ce droit, le serveur renvoie une erreur 403 Forbidden. La capture ci-dessous montre la mise en place complète pour un utilisateur de test.

```
bilelchakvm@bilelchak: ~  
exit  
bilelchakvm@bilelchak:~$ sudo  
-bash: sudo : commande introuvable  
bilelchakvm@bilelchak:~$ su -  
Mot de passe :  
root@bilelchak:~# a2enmod userdir  
Enabling module userdir.  
To activate the new configuration, you need to run:  
    systemctl restart apache2  
root@bilelchak:~# nano /etc/apache2/mods-available/php8.4.conf  
root@bilelchak:~# systemctl restart apache2  
root@bilelchak:~# useradd -m -s /bin/bash martin  
root@bilelchak:~# mkdir /home/martin/public_html  
root@bilelchak:~# nano /home/martin/public_html/index.php  
root@bilelchak:~# chown -R martin:martin /home/martin  
chown 755 /home/martin  
root@bilelchak:~# mysql_secure_installation  
-bash: mysql_secure_installation : commande introuvable  
root@bilelchak:~#
```

FIGURE 16 – Mise en place d’une page personnelle (création de l’utilisateur, public_html, droits 755).



FIGURE 17 – Page personnelle servie via UserDir : PHP s’exécute bien dans le dossier de l’utilisateur.

2.6 Base de données : administrateur et isolation

L’installation de MariaDB est d’abord sécurisée (suppression des comptes anonymes, de la base de test, etc.) :

mariadb-secure-installation

Un compte administrateur dédié est créé, plutôt que d'exposer le compte root du SGBD. Ce choix est purement sécuritaire : il évite d'exposer le super-utilisateur et limite les risques en cas de compromission d'une application web.

```
CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin';
GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

```
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin';
Query OK, 0 rows affected (0,012 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0,006 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,002 sec)

MariaDB [(none)]> EXIT;
Bye
root@bilelchak:~#
```

FIGURE 18 – Création du compte administrateur MariaDB avec tous les privilèges.

L'isolation est ensuite mise en place : chaque utilisateur possède une base portant son nom, sur laquelle il est le seul à avoir des droits. C'est l'emploi de `GRANT ALL PRIVILEGES ON login.*` (et non `*.*`) qui garantit qu'aucun utilisateur ne peut lire ou modifier la base d'un autre. La vérification se fait en se connectant en tant qu'utilisateur : `SHOW DATABASES` ne montre alors que sa propre base.

```
root@bilelchak:~# mariadb -u martin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 11.8.6-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| martin    |
+-----+
2 rows in set (0,000 sec)

MariaDB [(none)]> _
```

FIGURE 19 – Isolation validée : connecté en tant qu'utilisateur, `SHOW DATABASES` ne montre que sa base.

2.7 Scripts d'automatisation

Deux scripts sont placés dans `/usr/local/bin`. Ce répertoire faisant partie de la variable d'environnement `PATH`, les scripts deviennent appelables comme des commandes natives depuis n'importe où.

createDatabase

Ce script crée la base d'un utilisateur, son compte MySQL et ses privilèges. Un point technique important : le compte est créé pour DEUX hôtes.

```
#!/bin/bash
if [ "$(id -u)" -ne 0 ]; then echo "Erreur : a lancer en root."; exit 1; fi
if [ "$#" -ne 1 ]; then echo "Usage : createDatabase <login>"; exit 1; fi

login="$1"
mariadb <<SQL
CREATE DATABASE IF NOT EXISTS '$login';
CREATE USER IF NOT EXISTS '$login'@'localhost' IDENTIFIED BY '$login';
CREATE USER IF NOT EXISTS '$login'@'127.0.0.1' IDENTIFIED BY '$login';
GRANT ALL PRIVILEGES ON '$login'.* TO '$login'@'localhost';
GRANT ALL PRIVILEGES ON '$login'.* TO '$login'@'127.0.0.1';
FLUSH PRIVILEGES;
SQL
echo "[OK] Base '$login' creee."
```

La création du compte pour localhost ET 127.0.0.1 répond à un besoin concret : PHP/PDO se connecte par le socket Unix (hôte localhost), tandis que le driver JDBC de Java se connecte par TCP (hôte 127.0.0.1).

createUser

```
#!/bin/bash
# createUser <login> <uid> : cree un utilisateur complet sur le serveur
# - compte systeme (UID force pour le NFS)
# - base de donnees (via createDatabase) + tables Contacts.sql
# - espace web ~login (index.php + displayContacts.php)
# - programme DisplayContacts.java dans le HOME

# Dossier ou se trouvent les 3 fichiers fournis
TEMPLATES="/root/sae-templates"

# Verifier qu'on est root
if [ "$(id -u)" -ne 0 ]; then
    echo "Erreur : a lancer en root."
    exit 1
fi

# Verifier qu'on a bien 2 arguments : le login et l'UID
if [ "$#" -ne 2 ]; then
    echo "Usage : createUser <login> <uid>"
    exit 1
fi

login="$1"
uid="$2"

# Creer compte systeme Linux avec l'UID
useradd -m -u "$uid" -s /bin/bash "$login"
echo "$login:$login" | chpasswd

# Creer l'espace web et la page d'accueil
mkdir -p /home/$login/public_html

cat > /home/$login/public_html/index.php <<EOF
<?php
\fmt = new IntlDateFormatter('fr_FR', IntlDateFormatter::LONG, IntlDateFormatter::NONE);
\stoday = \fmt->format(time());
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"/><title>Accueil de $login</title></head>
<body>
    <h1>Bienvenue $login !</h1>
    <p>Nous sommes le <?=\stoday ?></p>
    <p><a href="displayContacts.php">Voir mes contacts</a></p>
</body>
</html>
EOF

# Copie des fichiers fournis
cp $TEMPLATES/displayContacts.php /home/$login/public_html/
cp $TEMPLATES/DisplayContacts.java /home/$login/

# Remplacer les placeholders #dbname# par le login dans les 2 fichiers
sed -i "s/#dbname#/$login/g" /home/$login/public_html/displayContacts.php
sed -i "s/#dbname#/$login/g" /home/$login/DisplayContacts.java

# Creer la base de donnees de l'utilisateur
createDatabase "$login"

# Injection des tables fournies dans sa base
mariadb "$login" < $TEMPLATES/Contacts.sql

# Appliquer les droits : tout le HOME appartient a l'utilisateur
chown -R $login:$login /home/$login

# Apache doit pouvoir traverser le HOME pour atteindre public_html (sinon ça marche pas)
chmod 755 /home/$login

echo "====="
echo " Utilisateur '$login' (UID $uid) cree avec succes."
echo " Web : http://192.168.1.1/~$login"
echo "====="
```

C'est le script principal (voir sur VMSEVEUR pour le script complet `cat /usr/local/bin/createUser`). Il prend en paramètres un login et un UID, et réalise l'ensemble des opérations de création d'un utilisateur :

- Création du compte système Linux, en forçant l'UID (paramètre) — nécessaire pour le NFS.
- Création de l'espace web `public_html` et génération de la page d'accueil `index.php` (bienvenue, date du jour, lien vers les contacts).
- Copie des fichiers fournis : `displayContacts.php` dans l'espace web, `DisplayContacts.java` dans le HOME.
- Remplacement des marqueurs `#dbname#` de ces deux fichiers par le login, via la commande `sed` (les fichiers fournis sont génériques ; cette substitution les personnalise).
- Appel à `createDatabase`, puis injection des tables fournies (`Contacts.sql`) dans la base.
- Application des droits : `chown` récursif sur le HOME et `chmod 755` (pour éviter le 403).

Les fichiers fournis (`Contacts.sql`, `displayContacts.php`, `DisplayContacts.java`) ont été transférés depuis l'hôte Windows par `scp` via le tunnel, puis extraits dans `/root/sae-templates/`.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindows

PS C:\Users\Bilel> scp -P 2222 "C:\Users\Bilel\Downloads\FichiersFournis (1).tgz" bilelchakvm@localhost:/tmp/FichiersFournis.tgz
bilelchakvm@localhost's password:
FichiersFournis (1).tgz          100% 2267      1.1MB/s   00:00
PS C:\Users\Bilel>
```

FIGURE 20 – Transfert des fichiers fournis depuis Windows vers le serveur, par `scp` via le tunnel.

```
root@bilelchak:~# scp -P 2222 "C:\Users\Bilel\Downloads\FichiersFournis (1).tgz" bilelchakvm@localhost:/tmp/FichiersFournis.tgz
ssh: connect to host localhost port 2222: Connection refused
scp: Connection closed
root@bilelchak:~# mkdir -p /root/sae-templates
root@bilelchak:~# tar -xzf /tmp/FichiersFournis.tgz -C /root/sae-templates
root@bilelchak:~# ls /root/sae-templates
Contacts.sql  DisplayContacts.java  displayContacts.php
root@bilelchak:~#
```

FIGURE 21 – Extraction des fichiers fournis dans `/root/sae-templates/` sur le serveur.

L'exécution du script sur un utilisateur de test crée l'ensemble en une seule commande :

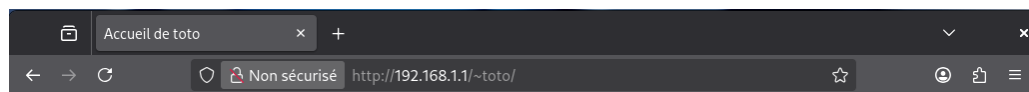
```

root@bilelchak:~# nano /usr/local/bin/createUser
root@bilelchak:~# chmod +x /usr/local/bin/createUser
root@bilelchak:~# cat /etc/passwd | grep "/home"
bilelchakvm:x:1000:1000:bilelchakvm,,,:/home/bilelchakvm:/bin/bash
martin:x:1001:1001::/home/martin:/bin/bash
createdbtest:x:1002:1002::/home/createdbtest:/bin/bash
root@bilelchak:~# createUser toto 1100
[OK] Base 'toto' creee.

=====
Utilisateur 'toto' (UID 1100) cree avec succes.
Web  : http://192.168.1.1/~toto
=====
root@bilelchak:~#

```

FIGURE 22 – Exécution de `createUser` : l'utilisateur est créé de bout en bout, base comprise.



Bienvenue toto !

Nous sommes le 15 juin 2026

[Voir mes contacts](#)

FIGURE 23 – Page d'accueil générée pour l'utilisateur (bienvenue, date du jour, lien contacts).

The screenshot shows a web browser window with the title 'Mes contacts'. The address bar displays 'http://192.168.1.1/~toto/displayContacts.php'. The page content includes a table of contacts and a timestamp.

Mes contacts

Nous sommes le 15 juin 2026, il est 13:27

Prénom	Nom	Tél. ou Bureau
Alain	COLMERAUER	04 75 58 09 22
Edsger	DIJKSTRA	04 90 64 21 77
Grace	HOPPER	04 75 41 08 46
William	JOY	01 01 19 70 19
Brian	KERNIGHAN	01 01 19 70 31
Donald	KNUTH	01 90 46 12 88
Leslie	LAMPORT	04 75 41 04 22
Ada	LOVELACE	04 75 41 04 23
Braam	MOLENAAR	01 66 27 99 38
Dennis	RITCHIE	01 01 19 70 21
Andrew	TANNENBAUM	04 75 41 04 22
Ken	THOMSON	01 01 19 70 23"
Linus	TORVALD	04 01 19 70 22
Le 15 juin 2026 à 13h27		

FIGURE 24 – Page `displayContacts.php` : la table `Contacts` est lue dans la base via PDO.

2.8 Partage NFS

Côté serveur, le paquet `nfs-kernel-server` est installé et l'arborescence `/home` est exportée via `/etc/exports`, en restreignant l'accès à la seule adresse du client :

```
/home    192.168.1.7(rw,sync,no_subtree_check)
```

Cette restriction à l'IP du client est un choix de sécurité : aucune autre machine du réseau ne peut accéder aux dossiers personnels des développeurs. On recharge ensuite la configuration et on vérifie que le partage est actif.

```
# /home    192.168.1.7(rw,sync,no_subtree_check)
root@bilelchak:~# nano /etc/exports
root@bilelchak:~# exportfs -ra
root@bilelchak:~# systemctl restart nfs-kernel-server
root@bilelchak:~# exportfs -v
/home    192.168.1.7(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,root_squash,no_all_squash)
root@bilelchak:~#
```

FIGURE 25 – Le partage NFS est actif : /home exporté en lecture/écriture vers le client 192.168.1.7.

Côté client, l'installation du paquet `nfs-common` s'est heurtée à une difficulté : la carte NAT du client ne fournissait pas d'accès Internet (les deux cartes ne pouvant être actives simultanément sous NetworkManager). Il était donc impossible de télécharger `nfs-common` directement.

```
etudiant@debianiutv:~$ nmcli device status
DEVICE  TYPE      STATE      CONNECTION
enp0s3  ethernet  connecté   Wired connection 1
lo       loopback  connecté (en externe)  lo
enp0s8  ethernet  déconnecté  --
etudiant@debianiutv:~$
```

FIGURE 26 – Difficulté : `enp0s3` (Internet) et `enp0s8` (serveur) ne sont pas actives en même temps.

La solution adoptée a consisté à télécharger le paquet et toutes ses dépendances sur le serveur (qui dispose d'Internet), puis à les transférer vers le client par `scp` via le réseau interne, et enfin à les installer localement.

```
# Sur le serveur (qui a Internet)
apt-get download nfs-common libevent-core-2.1-7t64 libnfsidmap1 rpcbind keyutils

# Sur le client (via le reseau interne)
scp -r bilelchakvm@192.168.1.1:/tmp/nfspkg /tmp/
sudo apt install /tmp/nfspkg/*.deb
```

```
root@bilelchak:~# cd /root
root@bilelchak:~# apt-get download nfs-common
Réception de : 1 http://deb.debian.org/debian trixie/main amd64 nfs-common a
amd64 1:2.8.3-1 [269 kB]
269 ko réceptionnés en 6s (42,9 ko/s)
W: Le téléchargement est effectué en dehors du bac à sable en tant que super
utilisateur car le fichier « /root/nfs-common_1%3a2.8.3-1_amd64.deb » n'est
pas accessible par l'utilisateur « _apt ». - pkgAcquire::Run (13: Permission
non accordée)
root@bilelchak:~# |
```

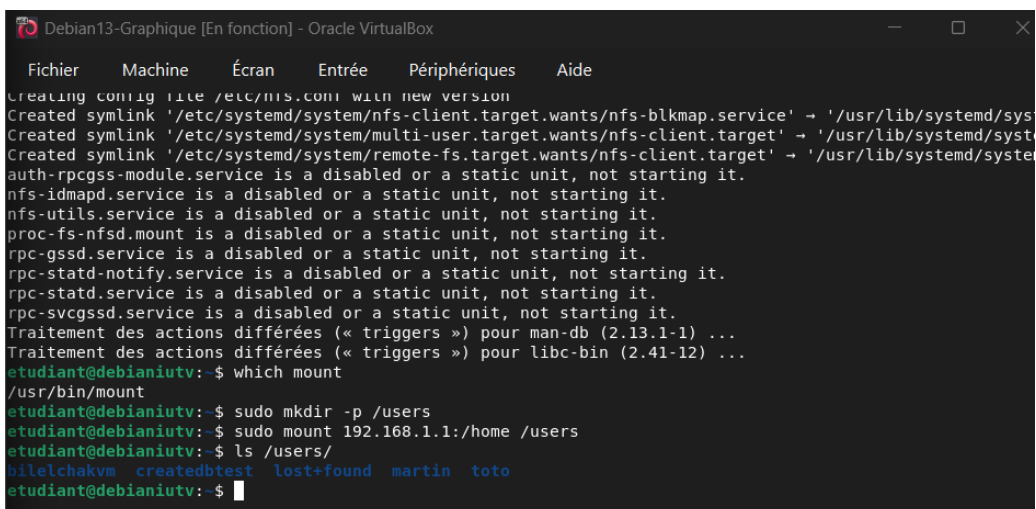
FIGURE 27 – Téléchargement de nfs-common sur le serveur, en vue d'un transfert vers le client.

```
etudiant@debianiutv:~$ dpkg -l | grep nfs-common
ii  nfs-common                                1:2.8.3-1
amd64 NFS support files common to client and server
etudiant@debianiutv:~$
```

FIGURE 28 – Vérification : le paquet nfs-common est bien installé sur le client (dpkg -l).

Le /home du serveur est ensuite monté sur le point de montage /users du client, et le montage est rendu permanent via /etc/fstab. Les utilisateurs du serveur apparaissent alors dans /users du client.

```
sudo mkdir -p /users
sudo mount 192.168.1.1:/home /users
# Permanence : ligne ajoutée dans /etc/fstab
192.168.1.1:/home /users nfs defaults 0 0
```



```
Debian13-Graphique [En fonction] - Oracle VirtualBox
Fichier  Machine  Écran  Entrée  Périphériques  Aide
creating config file /etc/nfs.conf with new version
Created symlink '/etc/systemd/system/nfs-client.target.wants/nfs-blkmap.service' → '/usr/lib/systemd/sys
Created symlink '/etc/systemd/system/multi-user.target.wants/nfs-client.target' → '/usr/lib/systemd/sys
Created symlink '/etc/systemd/system/remote-fs.target.wants/nfs-client.target' → '/usr/lib/systemd/syste
auth-rpcgss-module.service is a disabled or a static unit, not starting it.
nfs-idmapd.service is a disabled or a static unit, not starting it.
nfs-utils.service is a disabled or a static unit, not starting it.
proc-fs-nfsd.mount is a disabled or a static unit, not starting it.
rpc-gssd.service is a disabled or a static unit, not starting it.
rpc-statd-notify.service is a disabled or a static unit, not starting it.
rpc-statd.service is a disabled or a static unit, not starting it.
rpc-svcgssd.service is a disabled or a static unit, not starting it.
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.41-12) ...
etudiant@debianiutv:~$ which mount
/usr/bin/mount
etudiant@debianiutv:~$ sudo mkdir -p /users
etudiant@debianiutv:~$ sudo mount 192.168.1.1:/home /users
etudiant@debianiutv:~$ ls /users/
bilelchakvm  createdbtest  lost+found  martin  toto
etudiant@debianiutv:~$
```

FIGURE 29 – Montage NFS et rendu permanent (fstab) : le /home du serveur apparaît dans /users du client.

2.9 Liaison PHP / base de données (PDO)

Le fichier `displayContacts.php` fourni utilise PDO (PHP Data Objects) pour se connecter à MariaDB. Ce choix offre une couche d'abstraction propre et surtout une protection native contre les injections SQL grâce aux requêtes préparées. La connexion se fait en localhost : la base de données n'est jamais exposée sur le réseau, c'est Apache qui sert de pont sécurisé. L'affichage de la table des contacts (figure de la section 2.7) valide cette liaison de bout en bout.

2.10 Outils de développement Symfony et Composer

Composer (version 2.10.1) et Symfony CLI (version 5.17.1) ont été installés dans `/usr/local/bin`, afin d'être accessibles globalement par tous les développeurs du serveur sans réinstallation, comme l'exige le sujet.

2.11 Environnement Java

Le connecteur `libmariadb-java` est installé (il fournit `/usr/share/java/mariadb-java-client.jar`). Le programme `DisplayContacts.java` compile et s'exécute avec les commandes prévues par le sujet, et affiche la liste des contacts depuis la base de l'utilisateur. La connexion Java passant par TCP (127.0.0.1), ce test valide le compte MySQL `'login'@'127.0.0.1'` créé par `createDatabase`.

```
javac DisplayContacts.java
java -classpath /usr/share/java/mariadb-java-client.jar:. DisplayContacts
```

La sortie obtenue dans le terminal liste bien les 13 contacts de la table (Alain COLME-RAUER, Edsger DIJKSTRA, ...Linus TORVALD), ce qui confirme le bon fonctionnement de l'accès Java à la base.

3 Méthode de déploiement et tests de validation

3.1 Comment ajouter un nouvel utilisateur

La création d'un utilisateur suit le parcours imposé par le sujet, en deux temps, afin que son UID soit identique sur le client et sur le serveur — condition nécessaire au bon fonctionnement du partage NFS (qui transporte les numéros d'utilisateur, pas les noms).

Étape 1 — sur le client, on crée l'utilisateur et on relève son UID :

```
sudo useradd -m -u 1500 -s /bin/bash paul
sudo passwd paul
id -u paul          # -> 1500
```

Étape 2 — sur le serveur, on lance le script en forçant ce même UID :

```
createUser paul 1500
```

Le script effectue alors automatiquement toutes les opérations (compte système, base, espace web, fichiers, droits).

3.2 Tests de validation

Vérification de la cohérence des UID. Depuis le client, on vérifie que les fichiers de l'utilisateur, vus à travers le montage NFS (`/users/paul`), lui appartiennent bien. La présence de « paul paul » comme propriétaire et groupe confirme que l'UID est identique des deux côtés (c'est bon signe).

```
etudiant@debianiutv:~$ ls -l /users/paul/
total 12
-rw-rw-r-- 1 root root 2756 13 juin 16:48 DisplayContacts.class
-rw-r--r-- 1 paul paul 2377 13 juin 16:23 DisplayContacts.java
drwxrwxr-x 2 paul paul 4096 13 juin 16:26 public_html
etudiant@debianiutv:~$
```

FIGURE 30 – Les fichiers de `/users/paul` appartiennent bien à l'utilisateur paul (UID cohérent client/serveur).

Test d'édition via NFS. On crée un fichier depuis le client dans l'espace web de l'utilisateur (situé physiquement sur le serveur via NFS), et on vérifie qu'il est bien servi par Apache. Ce test boucle l'ensemble du workflow décrit par le sujet : un utilisateur du client édite ses fichiers du serveur, qui sont aussitôt accessibles via le web.

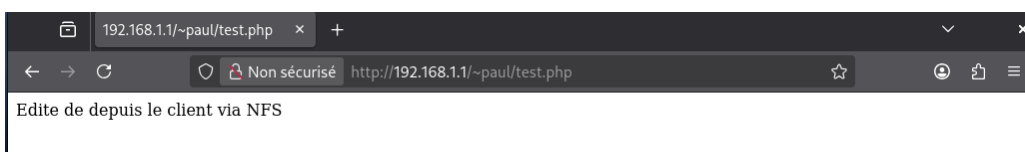


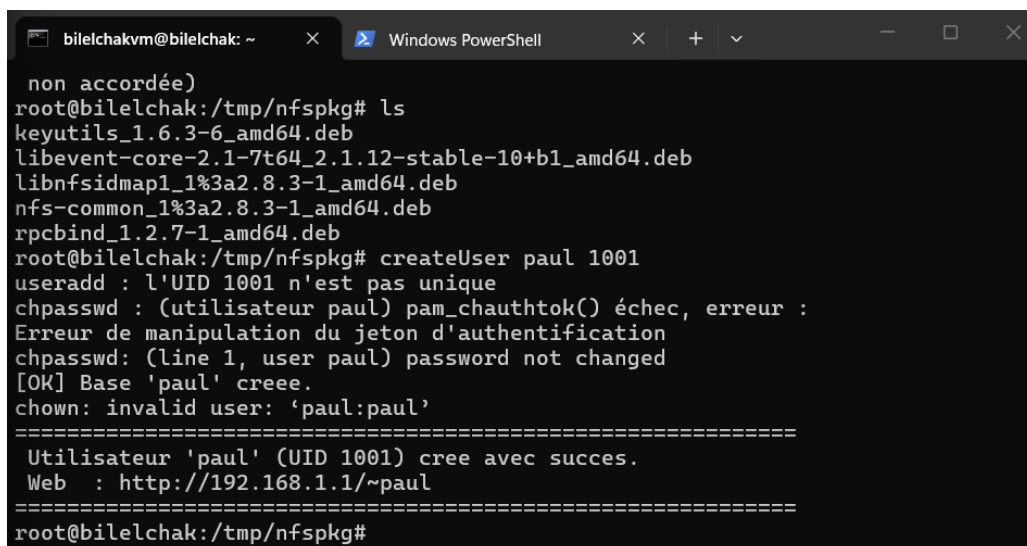
FIGURE 31 – Un fichier écrit depuis le client via NFS est immédiatement servi par Apache sur le serveur.

3.3 Difficultés rencontrées et solutions

Plusieurs difficultés ont été rencontrées et résolues au cours du projet. Les principales sont décrites ci-dessous.

Difficulté	Cause	Solution
Bureau installé par erreur	Validation trop rapide à l'étape tasksel	VM recrée proprement sans interface graphique
networking.service en échec (client)	NetworkManager gère le réseau sur Debian bureau	Configuration de l'IP via NetworkManager (IPv4 manuel)
Pas d'Internet sur le client	Carte NAT défaillante / cartes non simultanées	Téléchargement des .deb sur le serveur puis transfert par scp
Collision d'UID	UID 1001 déjà utilisé sur le serveur	Choix d'un UID libre des deux côtés (1500)
Erreur 403 sur les pages perso	Apache ne peut traverser le HOME	chmod 755 sur le répertoire personnel

La capture ci-dessous nous montre la collision d'UID : la création de l'utilisateur avec un UID déjà pris (1001) échoue, ce qui a conduit à choisir un UID libre.



```

non accordée)
root@bilelchak:/tmp/nfspkg# ls
keyutils_1.6.3-6_amd64.deb
libevent-core-2.1-7t64_2.1.12-stable-10+b1_amd64.deb
libnfsidmap1_1%3a2.8.3-1_amd64.deb
nfs-common_1%3a2.8.3-1_amd64.deb
rpcbind_1.2.7-1_amd64.deb
root@bilelchak:/tmp/nfspkg# createUser paul 1001
useradd : l'UID 1001 n'est pas unique
chpasswd : (utilisateur paul) pam_chauthtok() échec, erreur :
Erreur de manipulation du jeton d'authentification
chpasswd: (line 1, user paul) password not changed
[OK] Base 'paul' creee.
chown: invalid user: 'paul:paul'
=====
Utilisateur 'paul' (UID 1001) cree avec succes.
Web : http://192.168.1.1/~paul
=====
root@bilelchak:/tmp/nfspkg#

```

FIGURE 32 – Difficulté illustrée : la création avec un UID déjà utilisé (1001) échoue → choix d'un UID libre.

3.4 Conclusion et accès d'administration

À l'issue de ces opérations, le serveur est pleinement opérationnel et conforme au cahier des charges : serveur web Apache/PHP, bases MariaDB isolées par utilisateur, scripts d'automatisation, partage NFS, outils Symfony/Composer et accès Java fonctionnels. Le projet a permis de mettre en œuvre concrètement une architecture client-serveur complète et de comprendre les mécanismes qui la sous-tendent (réseau, droits, isolation, montage distant).

Identifiants d'administration de la base de données (requis dans les livrables) :

— Administrateur MariaDB : identifiant « **admin** », mot de passe « **admin** ».

- Pour chaque utilisateur créé : la base, le compte MySQL et le mot de passe portent son identifiant (ex. l'utilisateur paul a la base paul, accessible par paul / paul).

3.5 Accès aux machines

Cette section regroupe l'ensemble des accès nécessaires aux enseignants pour démarrer et tester la machine (clé USB / VM clonée). Le serveur est administré à distance via le tunnel SSH ; le client dispose d'une interface graphique.

Machine	Compte	Mot de passe
Serveur (système)	bilelchakvm	bilel
Serveur (root)	root	bilel
Client (système)	etudiant	iutval-info
Base de données (admin)	admin	admin

Connexion au serveur depuis la machine hôte Windows (via le tunnel) :

```
ssh -p 2222 bilelchakvm@localhost      # puis 'su -' pour passer root
```

La procédure de test (ajout d'un utilisateur et vérifications) est décrite à la section 3.1 et 3.2.